

Preemptive DoS attacks on Time Sensitive Networks

Nurefşan Sertbaş Bülbül and Mathias Fischer

Department of Informatics, Universität Hamburg, Germany

Email: {nurefsan.sertbas, mathias.fischer}@uni-hamburg.de

Abstract—Time-sensitive networking (TSN) is a promising technology for real-time communication in industrial and automotive networks. One of its key features is frame preemption, which allows high-priority traffic to interrupt the transmission of low-priority traffic, thereby reducing the delay of high-priority critical traffic. However, the deterministic nature of TSN frame preemption also makes it vulnerable to denial of service (DoS) attacks, which can severely impact flow quality of service (QoS) by increasing delays and packet loss. In this paper, we introduce the concept of preemptive DoS attacks and evaluate their impact on TSN QoS performance. We describe a strategy that attackers can use to estimate the preemption scheme configured in the switch and then demonstrate how an active attacker can use this information to degrade TSN QoS. Our simulation results indicate that even a single attacker can significantly deteriorate the QoS of TSN traffic. It is important to address this vulnerability in TSN and develop countermeasures to prevent preemptive DoS attacks from occurring.

Index Terms—TSN, frame preemption, time-sensitive networking, traffic analyzing attacks, calibrated attacks, denial of service

I. INTRODUCTION

Real-time communication has become increasingly critical in various industries, such as avionics, industrial, and automotive, where rapid and reliable data exchange is essential for safe and efficient operation. In these industries, traffic flows generated by time-critical (TC) applications with stringent requirements for bounded latency and low jitter often share communication channels with the flows originating from non-time-critical (NTC) applications. However, standard Ethernet does not provide any inherent guarantees for delivering time-sensitive data as it was not initially conceived for real-time applications and strict timing requirements.

In standard Ethernet, frames transmitted non-preemptively may experience significant queuing delays if there is no mechanism to interrupt transmissions. In the worst case, a high-priority TC frame may be delayed by a lower-priority NTC frame already being transmitted. Once the NTC frame has completed transmission, the TC frame will be processed as soon as possible. This may be fine for light traffic scenarios. However, for heavy traffic, long queuing delays might be the result. Using jumbo frames whose maximum transmission unit could be larger than 9k bytes can cause delays around 120 μ s per 100 Mbps to switch in the worst case [1], [2]. This is a considerable latency for TC traffic.

The IEEE Time-Sensitive Networking task group has proposed standards that enhance Ethernet with real-time capabilities and various QoS classes to address that problem. Particu-

larly, the IEEE 802.3br and IEEE 802.1Qbu standards propose a frame preemption mechanism allowing the preemption of certain traffic classes in favor of other classes [3], [4]. With that, frames configured as express can suspend transmission of the frames configured as preemptable. Hence, it enables processing and forwarding high-priority frames immediately upon arrival without being delayed by low-priority frames. Furthermore, it has been shown that with preemption, the waiting time for higher-priority packets becomes independent of the number of low-priority packets [5].

Although TSN frame preemption is crucial to TSN's real-time capability, it also creates potential security risks. It still suffers from worst-case delays that could be orders of magnitude longer than average, even when using only a single express traffic class [6]. Moreover, it can cause the starvation of NTC frames and exhaustion of full switch buffers [1]. These cases can occur spontaneously in a regular network and can be abused by attackers to target specific flows or, more generally, to increase the queueing delay. For instance, an attacker could send express traffic to the target switch just before the targeted traffic, causing long waiting times and a DoS. Therefore, the preemption mechanism requires further research, as it is a relatively new addition to the TSN family, and its vulnerability to preemptive DoS attacks needs to be thoroughly investigated.

This paper explores the vulnerabilities of TSN frame preemption and their implications for TSN networks. For that, an attacker first needs to passively monitor the network to extract useful information, such as the used configuration scheme in the preemption mechanism. Then, based on the observations, (s)he can attack the preemption mechanism. Accordingly, our contributions are:

- We identify various potential attack scenarios and discuss their possible impacts on network performance. In the process, we introduce the concept of preemptive DoS attacks.
- We demonstrate how even the passive observation of a TSN, e.g., monitoring a single input and output port of a switch, can be prolonged by attackers. Furthermore, with the OMNeT++ simulations, we show how feasible using the collected information to launch attacks is. Our simulation results indicate that even a single attacker can cause significant delays for specific traffic classes.

The rest of this paper is structured as follows: Section II briefly describes the frame preemption mechanism and describes the state of the art. In Section III, we examine

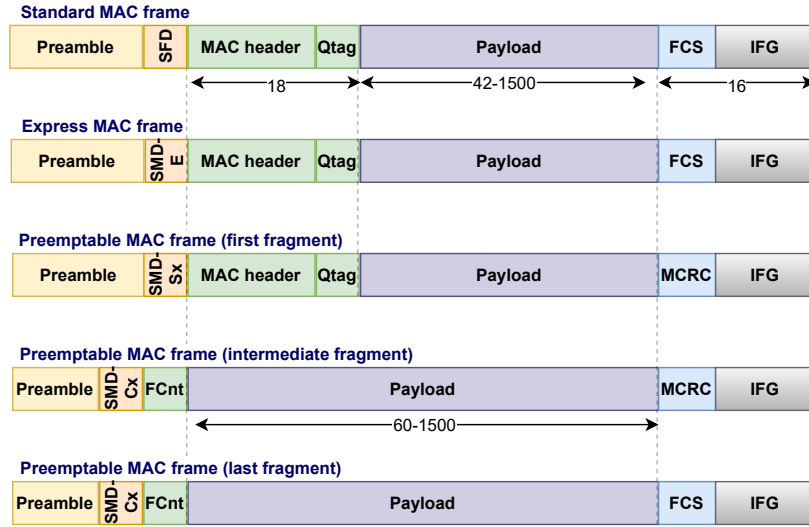


Fig. 1: MAC Frame Format

the vulnerabilities of the preemption mechanism and define threats. We give feasibility results in Section IV. Finally, Section V concludes the paper and summarizes future work.

II. BACKGROUND AND RELATED WORK

Frame preemption is a TSN mechanism that allows critical traffic to interrupt the lower criticality traffic transmission to satisfy strict timing requirements. Since preemption is defined on a transmission port of a switch, it requires changes both in the medium access control (MAC) layer and bridge management protocol. Accordingly, these two mechanisms are described in two standards: IEEE 802.1Qbu for bridge management and IEEE 802.3br for the Ethernet MAC.

At the egress port of the bridge, frames are handled differently depending on the configuration. The express MAC (eMAC) and preemptable MAC (pMAC) layers handle frames differently depending on whether they are configured as express or preemptable. Preemption can only occur between express and preemptable frames, and frames in the same class cannot preempt each other. Besides, frames of express classes cannot be preempted; they can only preempt preemptable frames. In preemption, the preempted frame's transmission is resumed until the express frame's transmission finishes. Therefore, it is ensured that high-priority traffic configured as express can no longer be delayed by low-priority preemptable traffic. The preempted frame can continue transmission from where it left off only after this point.

As a result of the preemption, preemptable frames are split into fragments. To make the frame preemption transparent to the Ethernet's physical layer, the basic MAC frame structure is preserved as shown in Fig. 1. Each frame starts with a preamble and ends with a cyclic redundancy check, FCS or MCRC, and an inter-frame gap (IFG). In express frame format, the start frame delimiter (SFD) field is replayed by SMD-E (start MFrame delimiter - express) to signal the express frame afterward. The first fragment of the preemptable

frame also has a very similar structure, signaling by SMD-Sx (SMD - start fragment) field. Following the first one, all fragments are signaled by SMD-Cx (SMD - continuation fragment) and a fragment counter (FCnt). This way, fragments can be combined at the MAC layer in order. Since IEEE 802.3br only allows one level of preemption, the MAC header and Q-Tag fields are transmitted once in the first fragment, allowing a bigger payload in the later fragments.

According to the standards, all MAC frames/fragments must meet the minimum Ethernet frame size requirement of 84 bytes. For the smaller fragments, the payload needs to be padded accordingly. However, padding is not allowed in the preemption, which results in some constraints to the payload sizes in the fragments. The smallest frame that can be preempted has a payload of 102 bytes, as it can be split into 42 bytes and 60 bytes to form a start and continuation fragment that fulfills the minimum Ethernet frame size. Considering the frame formats, the longest lower priority frame/fragment that can block an express frame is 143 bytes long, as proven in [2].

There have been several studies on the worst-case analysis of frame preemption and its effects on end-to-end transmission delays, as described in [1]. The authors tested a custom preemption mechanism in [7], and the implementation results confirm that latency and jitter of real-time traffic are reduced compared to standard priority-based Ethernet in the mixed traffic scenarios. For the practical use of frame preemption, an optimization framework was presented in [8] for assigning frames to preemptable or express queues and then deciding the optimal allocation of the queues as preemptable or express. The results of this study showed that the queuing configuration had a considerable impact on performance.

In [9], authors state a problem that an express frame can cause long delays over the preemptable frames. Also, since preemptable frames cannot preempt each other in some

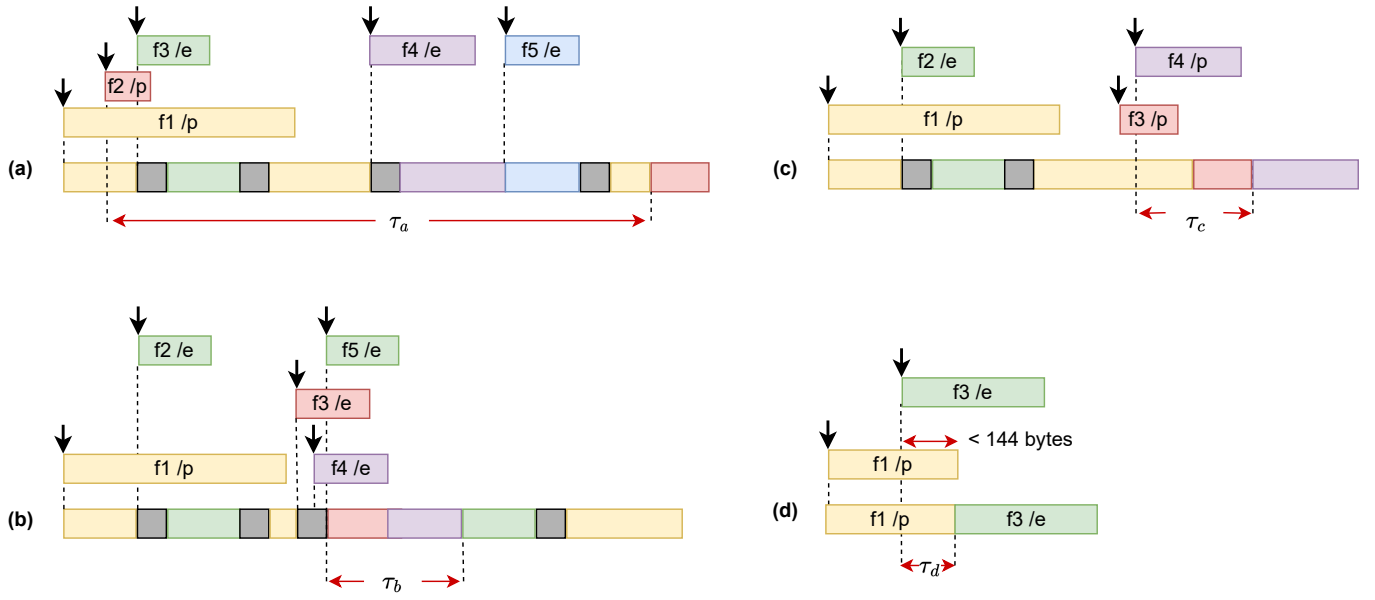


Fig. 2: Preemption effect on particular scenarios: (a) express frames to block preemptable frames, (b) express frames delaying each other, (c) preemptable frames delaying each other, (d) preemptable frames to block express frames

scenarios, huge delays may occur regardless of the timing constraints of preemptable frames. Thus, the authors propose a modification of the MAC layer by adding a third interface named tpMAC. So, express frames can preempt all classes, and frames assigned to tpMAC can also preempt preemptable frames. This way, the waiting time for the preemptable traffic classes with timing constraints can be shortened. The implementation aspects of this preemption mechanism are discussed in [10]. The authors focus mostly on implementation details and present a scalable architecture that can run in application-specific integrated circuits (ASIC) and field-programmable gate array (FPGA) platforms. Another solution to that problem is proposed in [11]. It requires modification of the pMAC and does not add a third interface as in [9]. Simulation results confirm that the presented mechanism achieves better response times for high-priority frames and only slightly impacts the response times of low-priority frames.

Recent work in [11] addresses the limitations of the current preemption mechanism. They claim that high-priority frames can experience significant blocking delays in certain cases, and they propose a credit-based preemption mechanism to lower the maximum blocking time of the express frames. However, to our knowledge, the preemption mechanism's vulnerabilities have yet to be discussed from the attacker's perspective in related work. This paper analyzes the limitations and malicious use of the standard frame preemption mechanism by simulations in a realistic TSN environment. Then, a few candidate solutions will be discussed to handle these limitations.

III. TSN FRAME PREEMPTION SCENARIOS

This section analyzes the preemption mechanism and lists its performance shortcomings for low and high-priority frames.

A. Preemption Effect on Particular Service Types

Frame preemption is already proposed to handle possible misuse of low-priority frames for targeting timing constraints of the high-priority frames. For that, high-priority express traffic is saved from the long waiting times due to low-priority traffic. However, even though it helps high-priority traffic preserve its timing constraints, frame preemption still has shortcomings. In the following, we list the effects of the different preemption cases on certain traffic classes.

Scenario-a: Express frames block preemptable frames:

The frame preemption mechanism can decrease significant waiting times of high-priority traffic. However, a preemptable frame can be preempted multiple times, which induces overhead as a part of the frame-splitting process that can significantly delay the transmission of preemptable traffic. This may also lead to the starvation of preemptable frames and the exhaustion of switch buffers. As can be seen in Fig. 2-a, even though the preemptable frame f_1 , arrives before the express frames, f_3, f_4, f_5 , it has to wait. As in this example, sending consecutive express frames results in multiple preemptions of f_1 and, therefore, long delays. Also, the other preemptable frame, f_2 , faces a similar long waiting time.

Scenario-b: Express frames delay each other: Even though express frames can not preempt each other, they can still cause delays. An attacker can send certain priority frames before the reception of a time-critical high-priority frame, as shown in Fig. 2-b. Suppose that green frames are periodic and the attacker has extracted that period. The attacker can insert higher priority frames before the next transmission of f_2 (so f_5). In that case, f_5 needs to wait for the transmissions of f_3 and f_4 . The attacker can adjust the number of inserted frames considering the extracted transmission period to delay

the transmission of f_5 and violate its timing constraints. Furthermore, in the worst case, all express streams in the network use the same egress port.

Scenario-c: Preemptable frames delay each other: According to the standard preemption mechanism, preemptable frames cannot preempt each other. If preemptable frames are forwarded in a first come, first served manner, as suggested in the standards, some low-priority but critical frames may suffer from long waiting times. Such frames are classified as preemptable but still may have softer timing constraints [12]. However, attackers can send a long preemptable frame before transmission to delay such lower-priority frames for longer periods. As can be seen in Fig. 2-c, if another preemptable frame f_3 , would be inserted just before the time critical but low priority preemptable frame f_4 . In this case, f_4 needs to wait until f_1 and f_3 have been transmitted.

Scenario-d: Preemptable frames block express frames: Even though neither express nor preemptable frames can preempt express frames, there are some exceptions in theory. As we mentioned in Section II, further preemption is not allowed if the fragment size does not meet the minimum Ethernet frame size requirement. Thus, the express frame needs to wait for the transmission of 143 bytes which is the longest non-preemptable size in the worst case, as can be seen in Fig. 2-d.

B. Configuration of Preemption Service Classes

Apart from the blocking possibilities of certain traffic services to each other, an improper configuration of these classes can also lead to threats. A higher-priority queue can be mapped to a pMAC, and a lower-priority queue can be mapped to an eMAC. Even though we do not expect such scenarios in practical implementations, improper or illegitimate configurations may occur.

IV. EVALUATION

In this section, we simulate a simple TSN network to show vulnerabilities of the standard frame preemption mechanism considering the defined attack scenarios presented in Section III. First, we briefly explain the evaluation setup and the attacker model. Then, we show the feasibility of the attacks on the TSN preemption mechanism. Finally, we summarize our simulation results.

A. Experimental Setup

We use OMNeT++ discrete event simulation framework for the simulative analysis with NeSTiNg simulation model developed for time-sensitive networks by extending inet library [13], [14]. For our experiments, we used a simple topology as shown in Fig. 3 in which we have two legitimate transmitting end-hosts per class, a *talker* in TSN, and one malicious *talker*, a single TSN switch and single receiving end-hosts, *listener* in TSN, as we only need to monitor the ingress and egress of a single switch. Also, we set the link capacity to 100 Mbps for simplicity. Finally, the experiments are repeated for 20 scenarios, and results are given with confidence intervals.

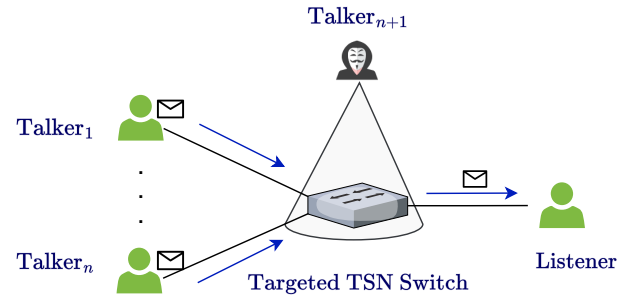


Fig. 3: Test topology

TABLE I: Simulated TSN traffic

	Type	Period	Deadline	Payload size
Q7	Control	10-40 ms	Same as period	10-100 Byte
Q6	Sensor	40-100 ms		100-200 Byte
Q5	CAM, Radar	20-100 ms		400-1500 Byte
Q0	Best effort	50-100 ms	-	1500 Byte

Since there is no publicly available data set for TSN traffic, we obtained the traffic generation parameters from TSN literature and tried to model TSN traffic as realistically as possible [8]. Considering in-vehicle networks as a use case, we generated four traffic classes; control signal, sensor data, raw data from camera or radar (lidar), and best-effort traffic, as shown in Table I. The best effort in this scenario does not require any timing guarantee and represents the non-time critical traffic. On the other hand, the remaining traffic classes are usually used by time-critical applications; thus, these data flows have specific traffic deadlines and represent time-critical traffic.

We also assume the preemption mechanism is appropriately configured as described in Section III-B. Hence, we assume low-priority queues are not configured as express when there is high-priority traffic in the network. We listed a few configuration scenarios in Table II. Here, P represents a related queue configured as preemptable, while E represents an express queue. For instance, in the third configuration, queues seven (Q7) and six (Q6) are configured as express, while others are configured as preemptable.

TABLE II: Possible frame preemption configurations

Configuration	Q7	Q6	Q5	Q0
#1	P	P	P	P
#2	E	P	P	P
#3	E	E	P	P
#4	E	E	E	P
#5	E	E	E	E

B. Evaluation Metrics

We used the following metrics to evaluate our scenarios:

- Observed Mean Variance in Delay: The mean-variance of the observed delay of either the applied scenario (e.g., class i traffic injection) or the initial status where no traffic is injected.

- Mean Stream Latency (MSL): The average latency of streams until they reach their destination.

C. Attacker Model

To show how the malicious talkers can abuse the preemption mechanism, the attack model needs to be defined, stating its capabilities. Here, two different attack models can be used as follows:

- *Attackers with full knowledge*: Attacker knows everything about the network. For instance, the attacker can calculate upper bounds for the queuing delay at each hop as (s)he knows about talkers' behavior.
- *Attackers with partial knowledge*: Here, the attacker can either (i) passively monitor the network and capture traffic or (ii) actively send certain priority probe packets to the network. Then, the attacker analyzes the time deltas between sent messages and their corresponding responses on specific links. Then, the attacker may perform traffic-analysis attacks to learn when a particular node forwards a specific frame or where it will be processed at the given time.

An attacker with full knowledge can manipulate the network easily due to the deterministic nature of TSN. However, it becomes challenging with limited knowledge. Thus, this paper assumes that the attacker initially has zero knowledge. Furthermore, we only assume that the switch configuration does not change during our observation period, which we select as $100\mu s$. During this period, the attacker monitors a series of messages at the ingress and egress of the target switch and sees the switch as a black box. Furthermore, the attacker can control a few talkers. In that case, it can inject traffic from a specific class to delay certain traffic flows.

D. Results

This section presents how an attacker can perform traffic analysis and how the scenarios defined in Section III become feasible based on the attacker's observations.

Traffic analysis: Assuming that the switch is a black box for an attacker, we only monitor the ingress and egress of the TSN switch and hold statistics per class, such as experienced frame delay. Thus, the attacker can perform traffic analysis by analyzing the relations between packets passing through the observed switch. For that, we record the arrival and departure times of the frames for all classes and extract minimum, maximum, and average latency and the variance of the latency values per class from the observed latency values. Preliminary results show that minimum, maximum, and average latency values differ significantly based on the generated traffic scenarios. However, relating these metrics with the used frame preemption configuration is hard.

Thus, we use the variance of the measured latency by injecting different classes of traffic and see how much it affects the latency of other classes (See Fig. 4). However, that does not require active traffic injection of the attacker; it may also be possible that the attacker just passively monitors the network where new talkers join the network and start sending

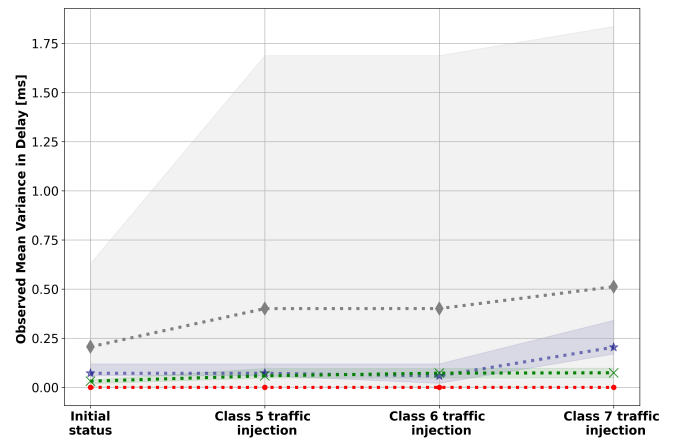


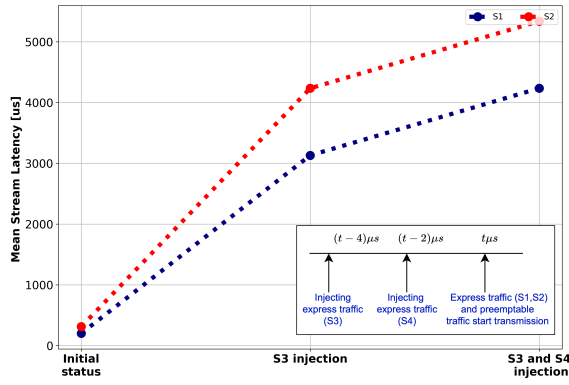
Fig. 4: Attacker observations in case of configuration 2 is used

traffic. Results of configuration #2, where only Q7 is set as an express, are shown in Fig. 4. It can be differentiated from the other classes independent of the injected traffic class; it is unaffected as it has a right to preempt other classes. In some cases where two express frames arrive simultaneously, we may see slight delays. However, this effect does not vary the results. Moreover, experiments using the other configurations in Table II resulted in parallel results and confirmed our claim that variance is an excellent metric for estimating the used configuration scheme, e.g., express traffic classes.

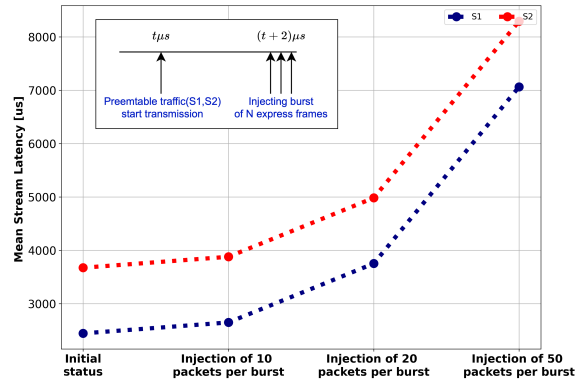
Injecting malicious traffic into the network: After the traffic analysis, the attacker can predict a specific traffic class assigned to a preemptable or an express. Then, using this information, it can perform the scenarios described in Section III. To show its feasibility, we simulate two scenarios and measure the success of the attacks in terms of the mean latency of the streams. Finally, we configure the Q7 as an express and the remaining queues as preemptable. For simplicity, we set the link capacity to 10 Mbps so that it is possible to see the preemption effect even with a small scenario.

In the first scenario, we generate two express streams, S1 and S2, and two preemptable streams. Then, we measure the mean end-to-end latency of express traffic as shown in Fig. 5a. Here, we left minimum and maximum values out as they are very narrow. Initially, we see that S1 and S2 have very low MSL, around $250\mu s$. Then, assuming the attacker passively monitored the network and detected S1 and S2 as express traffic, s(he) sends express traffic before S1 and S2 start transmission. Here, we set this value as $2\mu s$ so that, as shown in the timing diagram, $2\mu s$ before the transmission of S1 and S2, the attacker injects express traffic. Since express frames cannot preempt each other, we see a significant increase when malicious express traffic, S3, and S4, is injected into the network. The preliminary results show that it is possible to carry out a calibrated attack to delay the targeted streams further. In the worst case, this may cause a violation of QoS constraints, e.g., missing deadlines.

In the second scenario, to show how preemptable traffic is affected by the express traffic, we initially generate two



(a) Injecting express traffic to delay specific express stream(s)



(b) Injecting express traffic to delay specific preemptable stream(s)

Fig. 5: Injecting malicious traffic into the network

preemptable streams, S1 and S2. Then, we measure the mean end-to-end latency of preemptable traffic as shown in Fig. 5b. In case the attacker sends express frames during the transmission of preemptable frames, they can be preempted. For that, we inject express frames as bursts. Results claim that even a single express stream that generates multiple bursts periodically can cause a significant delay of preemptable traffic. In the worst case, preempting several times may cause the starvation of the preemptable traffic.

V. CONCLUSION

Frame preemption is one of the mechanisms in TSN that helps limit the waiting time of certain traffic classes so that strict QoS requirements can be satisfied. However, security has yet to be one of the main design concerns. Since the primary application domains of the TSN protocols are mission-critical systems, it is crucial to explore potential threats and apply candidate countermeasures. This paper presents a few vulnerabilities of the standard frame preemption mechanism and describes scenarios that malicious end hosts can abuse. Then, we simulate a typical TSN network and show how much useful information an attacker can extract and how this information can potentially be used to violate the QoS constraints of TSN traffic so denial of service.

In our future work, we intend to explore more intricate scenarios involving multiple switches to enhance the impact of attackers, even though a single switch can already cause significant delay. Additionally, to mitigate these attacks, we intend to deploy different strategies limiting the attacker's knowledge of the network using the moving target defense (MTD) concept. By introducing diversity into the network, MTD can randomize certain features and prevent attackers from predicting network traffic. This will make it harder for attackers to identify patterns and predict future events despite an increased attack budget. Specifically, we aim to make it difficult for attackers to determine when a particular flow is being forwarded by a specific TSN node or where it is being processed at any given time. These efforts are part of our broader goal to enhance the security of time-sensitive networks.

REFERENCES

- [1] W.-K. Jia, G.-H. Liu, and Y.-C. Chen, "Performance evaluation of IEEE 802.1 Qbu: Experimental and simulation results," in *38th Annual IEEE Conference on Local Computer Networks*. IEEE, 2013, pp. 659–662.
- [2] D. Thiele and R. Ernst, "Formal worst-case performance analysis of time-sensitive Ethernet with frame preemption," in *2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2016, pp. 1–9.
- [3] "IEEE Standard for Ethernet Amendment 5: Specification and Management Parameters for Interspersing Express Traffic," *IEEE Std 802.3br-2016 (Amendment to IEEE Std 802.3-2015 as amended by IEEE Std 802.3bw-2015, IEEE Std 802.3by-2016, IEEE Std 802.3bq-2016, and IEEE Std 802.3bp-2016)*, pp. 1–58, 2016.
- [4] "IEEE Standard for Local and Metropolitan Area Networks – Bridges and Bridged Networks – Amendment 26: Frame Preemption," *IEEE Std 802.1Qbu-2016 (Amendment to IEEE Std 802.1Q-2014)*, 2016.
- [5] Z. Zhou, Y. Yan, S. Ruepp, and M. Berger, "Analysis and implementation of packet preemption for time sensitive networks," in *2017 IEEE 18th International Conference on High Performance Switching and Routing (HPSR)*. IEEE, 2017, pp. 1–6.
- [6] D. Hellmanns, J. Falk, A. Glavackij, R. Hummen, S. Kehrler, and F. Dürr, "On the Performance of Stream-based, Class-based Time-aware Shaping and Frame Preemption in TSN," in *2020 IEEE International Conference on Industrial Technology (ICIT)*, 2020, pp. 298–303.
- [7] J. Kim, B. Y. Lee, and J. Park, "Preemptive switched ethernet for real-time process control system," in *2013 11th IEEE International Conference on Industrial Informatics*. IEEE, 2013, pp. 171–176.
- [8] T. Park, S. Samii, and K. G. Shin, "Design optimization of frame preemption in real-time switched ethernet," in *2019 Design, Automation and Test in Europe Conference (DATE)*. IEEE, 2019, pp. 420–425.
- [9] M. A. Ojewale, P. M. Yomsi, and B. Nikolić, "Multi-Level Preemption in TSN: Feasibility and Requirements Analysis," in *IEEE 23rd International Symposium on Real-Time Distributed Computing (ISORC)*, 2020.
- [10] M. Knezic, M. Kovacevic, and Z. Ivanovic, "Implementation Aspects of Multi-Level Frame Preemption in TSN," in *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, vol. 1, 2020, pp. 1127–1130.
- [11] M. Ashjaei, M. Sjödin, and S. Mubeen, "A novel frame preemption model in TSN networks," *Journal of Systems Architecture*, vol. 116, p. 102037, 2021.
- [12] M. A. Ojewale, P. M. Yomsi, and B. Nikolić, "Multi-level preemption in TSN: feasibility and requirements analysis," in *2020 IEEE 23rd International Symposium on Real-Time Distributed Computing (ISORC)*. IEEE, 2020, pp. 47–55.
- [13] J. Falk, D. Hellmanns, B. Carabelli, N. Nayak, F. Dürr, S. Kehrler, and K. Rothermel, "NeSTiNg: Simulating IEEE time-sensitive networking in OMNeT++," in *Proceedings of the 2019 International Conference on Networked Systems*, Garching b. München, Germany, Mar. 2019.
- [14] "INET framework." [Online]. Available: <https://inet.omnetpp.org/>